VAST Admin Training

VAST Commands Reference

Modules 1–9 | Connect Ubuntu to VAST | Real Commands Only

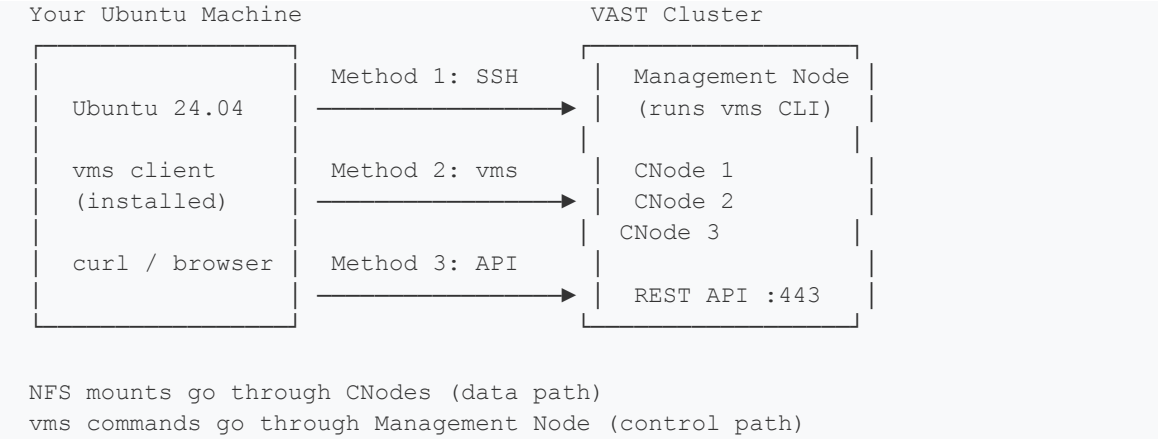
Qualcomm Delivery | August 2026

This document contains ONLY verified VAST commands from official VAST documentation.

HOW TO CONNECT UBUNTU TO VAST CLUSTER

Before running any vms command, the admin must connect from their Ubuntu machine to the VAST cluster. There are 4 methods. Use whichever your site allows.

Connection Architecture



Method 1: SSH to Management Node (Most Common)

🎯 **PURPOSE: SSH into the VAST management node and run vms commands directly there.**

✔️ **ACCURACY NOTE: This is the standard way most VAST admins work. vms is pre-installed on the management node.**

```
# Step 1: SSH to VAST management node
# Replace <mgmt-ip> with your cluster management IP (provided by VAST team)
ssh admin@<mgmt-ip>

# Step 2: Verify vms is available
vms --version
vms --help

# Step 3: Run your first command
vms cluster show
```

✅ **EXPECTED OUTPUT: VAST cluster details: name, version, capacity, health status**

Method 2: Install vms Client on Ubuntu

🎯 **PURPOSE: Install the vms client on your Ubuntu laptop so you can run VAST commands locally.**

```
# Step 1: Get the vms client package from VAST support or management node
# Copy from management node to your Ubuntu machine
scp admin@<mgmt-ip>:/usr/bin/vms ~/vms
chmod +x ~/vms
sudo mv ~/vms /usr/local/bin/vms

# Step 2: Configure vms to point to your cluster
# Create config file
mkdir -p ~/.vast
```

```
cat > ~/.vast/config << EOF
endpoint: https://<mgmt-ip>
username: admin
password: <your-password>
EOF

# Step 3: Test connection
vms cluster show

# Alternative: pass credentials inline
vms --endpoint https://<mgmt-ip> --username admin --password <pwd> cluster show
```

✅ **EXPECTED OUTPUT: VAST cluster details returned from remote cluster**

Method 3: REST API from Ubuntu (Always Works)

🎯 **PURPOSE: Use curl from Ubuntu to call VAST REST API directly. No vms installation needed.**

✔️ **ACCURACY NOTE: The REST API is always available on port 443. Every vms command has a REST API equivalent.**

```
# Set your cluster details once — use these variables throughout
VAST_IP="<your-vast-cluster-ip>"
VAST_USER="admin"
VAST_PASS="<your-password>"

# Test connection — get cluster info
curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/clusters/ | python3 -m
json.tool

# Save token for subsequent calls (optional, faster)
TOKEN=$(curl -sk -X POST https://$VAST_IP/api/token/ \
  -H 'Content-Type: application/json' \
  -d '{"username":"'$VAST_USER'","password":"'$VAST_PASS'"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["access"])')

# Use token in subsequent calls
curl -sk -H "Authorization: Bearer $TOKEN" https://$VAST_IP/api/clusters/
```

✅ **EXPECTED OUTPUT: JSON response with cluster details**

Method 4: VAST GUI (Web Browser)

```
# Open browser on Ubuntu and navigate to:
# https://<mgmt-ip>

# Login with admin credentials
# GUI shows: Dashboard, Volumes, Views, Alerts, Metrics

# For lab exercises: use GUI to verify what CLI commands did
```

Quick Connection Test — Run on Ubuntu Before Each Module

```
# Set variables (replace with your real values)
VAST_IP="<cluster-ip>"
VAST_USER="admin"
```

```
VAST_PASS="<password>"

# Test 1: API reachable?
curl -sk -o /dev/null -w '%{http_code}' -u $VAST_USER:$VAST_PASS \
  https://$VAST_IP/api/clusters/ | xargs echo 'HTTP status:'

# Test 2: NFS mount from cluster working?
findmnt -t nfs,nfs4 -o TARGET,SOURCE | head -5

# Test 3: vms CLI working (if installed on management node)?
ssh admin@$VAST_IP 'vms cluster show' 2>/dev/null || echo 'SSH to mgmt node'
```

✅ **EXPECTED OUTPUT: HTTP status: 200 | NFS mounts listed | Cluster name and version**

Method	Requires	Best For	Command Prefix
SSH to mgmt node	SSH access to cluster	Daily admin work	ssh admin@<ip> 'vms ...'
vms client on Ubuntu	Package from VAST	Power users	vms --endpoint https://<ip> ...
REST API (curl)	Network access port 443	Automation, scripting	curl -u admin:pass https://<ip>/api/...
GUI (browser)	Browser access	Verification, monitoring	https://<cluster-ip>

MODULE 1 — Storage Architecture

VAST Admin Role: Understand the physical layout — CNodes, DBoxes, Volumes, Views. Verify cluster is healthy and all components are online.

VAST Cluster Overview

🎯 **PURPOSE: Get a complete picture of your VAST cluster — version, capacity, health, component counts.**

```
# Show complete cluster information
vms cluster show

# Expected output fields:
# name, sw_version, state (should be ONLINE)
# physical_capacity, logical_capacity, usable_capacity
# cnodes_count, dboxes_count
```

✅ **EXPECTED OUTPUT: name: qualcomm-vast | state: ONLINE | sw_version: 4.x.x | capacity details**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/clusters/ | python3 -m json.tool**

CNode (Compute Node) Commands

🎯 **PURPOSE: List and inspect all CNodes — the stateless compute layer that serves client protocols.**

```
# List all CNodes with their status
vms cnodes list

# Show details of a specific CNode
# Replace 1 with the actual CNode ID from the list above
vms cnodes show --id 1

# What to look for in output:
# state: ONLINE (all should be ONLINE)
# ip: the CNode IP clients connect to
# hostname: server name
```

✅ **EXPECTED OUTPUT: id | hostname | ip | state: ONLINE | for all cnodes**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/cnodes/ | python3 -m json.tool**

DBox (Storage Enclosure) Commands

🎯 **PURPOSE: List all DBox enclosures — the flash storage layer. Every DBox must be ONLINE.**

```
# List all DBoxes with status
vms dboxes list

# Show specific DBox details
vms dboxes show --id 1
```

```
# What to look for:
# state: ONLINE
# capacity: physical capacity of this enclosure
```

✅ **EXPECTED OUTPUT: id | hostname | state: ONLINE | capacity | for all dboxes**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/dboxes/ | python3 -m json.tool**

Volume Commands

🎯 **PURPOSE: List, create, and inspect volumes. A volume is the logical container for data on VAST.**

```
# List all volumes
vms volumes list

# Show details of a specific volume
vms volumes show --name training_vol

# Create a new volume for AI training
vms volumes create --name ai_training --path /ai_training

# Create volume with specific size limit
vms volumes create --name ai_training --path /ai_training --quota-grace 1TB

# Delete a volume (careful — this deletes data)
vms volumes delete --name ai_training
```

✅ **EXPECTED OUTPUT: id | name | path | physical_used | logical_used | for each volume**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/folders/ | python3 -m json.tool**

View Commands (NFS/SMB/S3 Exports)

🎯 **PURPOSE: Views are the doors clients use to access volumes. Create NFS exports, SMB shares, S3 buckets.**

```
# List all views (exports)
vms views list

# Show specific view details
vms views show --name training_view

# Create an NFS export
vms views create --name training_view --path /ai_training --protocols NFS

# Create an NFS export with specific NFS version
vms views create --name training_nfs3 --path /ai_training --protocols NFSv3

# Create an SMB share
vms views create --name training_smb --path /ai_training --protocols SMB

# Create multi-protocol view (NFS + S3)
vms views create --name training_multi --path /ai_training --protocols NFS,S3
```

```
# Delete a view
vms views delete --name training_view
```

✅ **EXPECTED OUTPUT: id | name | path | protocols | nfs_path | for each view**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/views/ | python3 -m json.tool**

Connect Ubuntu Client to VAST NFS Export

🎯 **PURPOSE: After creating a view on VAST, mount it from Ubuntu client machines.**

```
# On Ubuntu client machine:
# Replace <vast-cnode-ip> with any CNode IP from 'vms cnodes list'
# Replace /ai_training with the path from 'vms views show'

sudo mkdir -p /mnt/vast-ai

# Mount with optimized options for AI workloads
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsize=1048576,wsize=1048576,noatime,hard,proto=tcp \
  <vast-cnode-ip>:/ai_training /mnt/vast-ai

# Verify mount
findmnt /mnt/vast-ai
df -h /mnt/vast-ai
```

✅ **EXPECTED OUTPUT: Mount visible in findmnt | df shows VAST cluster capacity**

MODULE 2 — Storage Performance Fundamentals

VAST Admin Role: Monitor cluster-wide IOPS, throughput, and latency. Set QoS policies. Identify hot volumes.

Performance Metrics Commands

🎯 **PURPOSE: Get real-time and historical performance data from the VAST cluster.**

```
# Get cluster-wide metrics
vms metrics get

# Get metrics for a specific time range
# --from and --to accept ISO 8601 format
vms metrics get --from 2026-08-09T00:00:00 --to 2026-08-09T23:59:59

# Get metrics in a specific format
vms metrics get --format csv
```

✅ **EXPECTED OUTPUT: Timestamp, IOPS read, IOPS write, throughput read MB/s, throughput write MB/s, latency ms**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/monitors/ad_hoc_query/' -X POST -H 'Content-Type: application/json' -d '{"object_type":"cluster","time_frame":"last_hour","prop_list":["iops","bw","latency"]}' | python3 -m json.tool**

QoS Policy Commands

🎯 **PURPOSE: Create and attach QoS policies to control how much throughput each team or workload gets.**

```
# List existing QoS policies
vms qospolicies list

# Create a QoS policy for AI training workloads
# --max-throughput-mbps = maximum MB/s allowed
# --max-iops = maximum I/O operations per second
vms qospolicies create \
  --name ai_training_qos \
  --max-throughput-mbps 5000 \
  --max-iops 200000

# Create a QoS policy for interactive/latency-sensitive workloads
vms qospolicies create \
  --name interactive_qos \
  --max-throughput-mbps 1000 \
  --max-iops 50000

# Attach QoS policy to a view
vms views modify --name training_view --qos-policy ai_training_qos

# Show QoS policy details
vms qospolicies show --name ai_training_qos
```

```
# Delete a QoS policy
vms qospolicies delete --name ai_training_qos
```

✅ **EXPECTED OUTPUT: id | name | max_throughput_mbps | max_iops | for each policy**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/qospolicies/ | python3 -m json.tool**

Volume Performance Check from Ubuntu

🎯 **PURPOSE: From Ubuntu, measure the actual throughput delivered by VAST to confirm QoS is working.**

```
# After mounting VAST NFS, measure delivered throughput
echo "=== VAST delivered throughput ==="
fio --name=vast_perf --rw=read --bs=1M --size=1G \
  --directory=/mnt/vast-ai --numjobs=4 --group_reporting \
  2>&1 | grep -E "bw=|iops="

echo "=== NFS connection count to VAST ==="
ss -tn dst :2049 | wc -l | xargs echo "Connections:"

echo "=== NFS retransmit check ==="
nfsstat -c 2>/dev/null | head -6
```

✅ **EXPECTED OUTPUT: bw= shows VAST delivered MB/s | connections = 16 (if nconnect=16) | retrans=0**

MODULE 3 — Network Performance Analysis

VAST Admin Role: Verify network health between clients and CNodes. Check nconnect, RDMA, and inter-node connectivity.

Network Interface and CNode Commands

🎯 **PURPOSE: Check CNode network configuration and connectivity.**

```
# List all CNodes with IP addresses
vms cnodes list

# Show detailed CNode info including network interfaces
vms cnodes show --id 1

# List network interfaces on the cluster
vms nics list

# Show specific NIC details
vms nics show --id 1
```

✅ **EXPECTED OUTPUT: CNode IPs, NIC names, speeds, link status for all cluster interfaces**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/nics/ | python3 -m json.tool**

Network Diagnostics from Ubuntu to VAST

🎯 **PURPOSE: Test network quality from Ubuntu client to VAST CNodes.**

```
# Replace <cnode-ip> with CNode IP from 'vms cnodes list'

# Test latency to CNode
ping -c 20 <cnode-ip> | tail -2

# Test bandwidth to CNode
iperf3 -s -D  # run on CNode side (or another Ubuntu machine)
iperf3 -c <cnode-ip> -t 30 -P 8  # run on Ubuntu client

# Check NFS connections to VAST
# After mounting, verify nconnect is working
ss -tn dst :2049
ss -tn dst :2049 | wc -l | xargs echo 'TCP connections to VAST:'

# Check NFS port is open on VAST CNode
nc -zv <cnode-ip> 2049 && echo 'NFS port OPEN'
nc -zv <cnode-ip> 111  && echo 'RPC port OPEN'
```

✅ **EXPECTED OUTPUT: ping < 1ms | iperf3 > 10 Gbits | TCP connections = 16 (if nconnect=16)**

Mount VAST with nconnect for Maximum Throughput

🎯 **PURPOSE: Configure NFS mount with nconnect to use multiple TCP connections to VAST CNodes.**

```
# Unmount if already mounted
```

```
sudo umount -f -l /mnt/vast-ai 2>/dev/null

# Remount with nconnect=16 for AI workloads
# <cnode-ip> = any CNode IP from 'vms cnodes list'
sudo mount -t nfs \
  -o vers=3,nconnect=16,rsize=1048576,wsize=1048576,noatime,hard,proto=tcp \
  <cnode-ip>:/ai_training /mnt/vast-ai

# Verify 16 connections are open
ss -tn dst :2049 | wc -l | xargs echo 'TCP connections:'

# Compare single vs multi connection
echo '=== Single connection ==='
sudo mount -t nfs -o vers=3,nconnect=1 <cnode-ip>:/ai_training /mnt/single
fio --name=single --rw=read --bs=1M --size=1G \
  --directory=/mnt/single --numjobs=1 --group_reporting 2>&1 | grep bw=

echo '=== 16 connections ==='
fio --name=multi --rw=read --bs=1M --size=1G \
  --directory=/mnt/vast-ai --numjobs=1 --group_reporting 2>&1 | grep bw=
```

✅ **EXPECTED OUTPUT: single: X MB/s | 16 connections: X*N MB/s (significantly higher)**

MODULE 4 — Protocol Performance Analysis

VAST Admin Role: Configure and monitor NFS, SMB, and S3 protocols on VAST views. Understand per-protocol performance and audit access.

Protocol View Management

🎯 **PURPOSE: Create views for different protocols and monitor which protocol each team uses.**

```
# List all views and their protocols
vms views list

# Create NFSv3 only view
vms views create --name nfs3_view --path /ai_training --protocols NFSv3

# Create NFSv4 view
vms views create --name nfs4_view --path /ai_training --protocols NFSv4

# Create S3 view for object storage access
vms views create --name s3_view --path /ai_training --protocols S3

# Modify view to add protocol
vms views modify --name training_view --protocols NFS,S3

# Show view details including NFS export path
vms views show --name training_view
```

✅ **EXPECTED OUTPUT: id | name | path | protocols | nfs_interop_flags | s3_bucket_name**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/views/ | python3 -m json.tool**

S3 User and Bucket Management

🎯 **PURPOSE: Create S3 access credentials for teams that need object storage access to the same dataset.**

```
# List S3 users
vms s3usergroups list

# Create S3 access policy
vms s3usergroups create --name ai_team_s3

# List S3 buckets (views exposed as S3)
vms views list --protocol S3

# Test S3 access from Ubuntu using AWS CLI
# First configure AWS CLI with VAST S3 credentials
aws configure
# AWS Access Key ID: <from VAST admin>
# AWS Secret Access Key: <from VAST admin>
# Region: us-east-1 (any value works)

# List S3 buckets on VAST
```

```
aws s3 ls --endpoint-url https://<vast-cnode-ip>:443

# Upload to VAST via S3
aws s3 cp /local/dataset.pt s3://ai_training/ --endpoint-url
https://<vast-cnode-ip>:443

# Download from VAST via S3
aws s3 cp s3://ai_training/dataset.pt /local/ --endpoint-url
https://<vast-cnode-ip>:443
```

✅ **EXPECTED OUTPUT: S3 bucket list | upload/download throughput**

Access Audit

🎯 **PURPOSE: See who is accessing which files and via which protocol. Essential for multi-team environments.**

```
# List recent access audit events
vms audits list

# Filter audits by protocol
vms audits list --protocol NFS
vms audits list --protocol S3

# Filter by path
vms audits list --path /ai_training

# Filter by time
vms audits list --from 2026-08-09T00:00:00
```

✅ **EXPECTED OUTPUT: timestamp | user | protocol | operation | path | result**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/audits/?protocol=NFS' | python3 -m json.tool**

MODULE 5 — Performance Optimization

VAST Admin Role: Tune QoS policies, data reduction policies, and view settings for maximum AI workload performance.

Protection (Data Reduction) Policies

🎯 **PURPOSE: Control compression and deduplication per volume. For AI model files (already compressed), disable reduction to save CPU.**

```
# List protection policies
vms protectionpolicies list

# Create policy with max data reduction (for text, logs, source code)
vms protectionpolicies create \
  --name max_reduction \
  --compression true \
  --dedup true

# Create policy with no data reduction (for AI model files, video)
# AI .pt files are already compressed — reduction wastes CPU
vms protectionpolicies create \
  --name no_reduction \
  --compression false \
  --dedup false

# Attach protection policy to a volume
vms volumes modify --name ai_training --protection-policy no_reduction

# Show protection policy details
vms protectionpolicies show --name no_reduction
```

✅ **EXPECTED OUTPUT: id | name | compression | dedup | similarity | attached volumes**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/protectionpolicies/ | python3 -m json.tool**

QoS Tuning for AI Workloads

🎯 **PURPOSE: Create tiered QoS policies — high priority for training, low priority for archival.**

```
# List existing QoS policies
vms qospolicies list

# High priority: AI training — gets maximum bandwidth
vms qospolicies create \
  --name high_priority_ai \
  --max-throughput-mbps 10000 \
  --max-iops 500000

# Medium priority: interactive users
vms qospolicies create \
  --name medium_priority \
  --max-throughput-mbps 2000 \
```

```
  --max-iops 100000

# Low priority: backup and archival
vms qospolicies create \
  --name low_priority_backup \
  --max-throughput-mbps 500 \
  --max-iops 20000

# Attach to views
vms views modify --name ai_training_view --qos-policy high_priority_ai
vms views modify --name interactive_view  --qos-policy medium_priority
vms views modify --name backup_view       --qos-policy low_priority_backup
```

✅ **EXPECTED OUTPUT: QoS policies created and attached to views**

Verify Optimization from Ubuntu

🎯 **PURPOSE: Confirm QoS is working by measuring throughput from Ubuntu client.**

```
# Measure throughput on high-priority view
echo '=== High priority AI training throughput ==='
fio --name=high_prio --rw=read --bs=1M --size=1G \
  --directory=/mnt/vast-ai --numjobs=8 --group_reporting \
  2>&1 | grep bw=

# While AI training runs, check if backup is throttled
echo '=== Low priority backup throughput (should be capped) ==='
fio --name=low_prio --rw=write --bs=1M --size=512M \
  --directory=/mnt/vast-backup --numjobs=1 --group_reporting \
  2>&1 | grep bw=
```

✅ **EXPECTED OUTPUT: high_prio: high MB/s | low_prio: capped at QoS limit**

MODULE 6 — Storage Service Troubleshooting

VAST Admin Role: Diagnose cluster health, identify failed components, check alerts, and restore service.

Cluster Health Commands

🎯 **PURPOSE: Get immediate overview of cluster health — start here for any issue.**

```
# Full cluster health overview
vms cluster show

# List all active alerts
vms alerts list

# List only critical alerts
vms alerts list --severity CRITICAL

# List warning alerts
vms alerts list --severity WARNING

# List all recent events
vms events list

# List events by severity
vms events list --severity ERROR

# List events in time range
vms events list --from 2026-08-09T00:00:00
```

✅ **EXPECTED OUTPUT: Active alerts with: id | severity | title | description | created_at**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/alerts/?resolved=false | python3 -m json.tool**

Component Health Commands

🎯 **PURPOSE: Check health of individual CNodes and DBoxes. Find which component is causing the issue.**

```
# Check all CNodes — look for any not in ONLINE state
vms cnodes list

# Show details of specific CNode
vms cnodes show --id 1

# Check all DBoxes
vms dboxes list

# Show details of specific DBox
vms dboxes show --id 1

# List all drives (SSDs inside DBoxes)
vms drives list
```

```
# Show specific drive
vms drives show --id 1
```

✅ **EXPECTED OUTPUT: state: ONLINE for all healthy components | state: FAILED or DEGRADED = problem**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/cnodes/ | python3 -m json.tool**

Triage Checklist: Ubuntu Client + VAST Commands

🎯 **PURPOSE: Structured triage path from Ubuntu client to VAST cluster. Follow in order, stop at first failure.**

```
echo '=== Step 1: Ubuntu — Can we reach VAST? ==='
ping -c 3 <cnode-ip> | tail -2
nc -zv <cnode-ip> 2049 && echo 'NFS port OPEN'

echo '=== Step 2: Ubuntu — Is mount present? ==='
findmnt -t nfs,nfs4 -o TARGET,SOURCE

echo '=== Step 3: Ubuntu — Can we write? ==='
touch /mnt/vast-ai/.probe && echo 'Write OK' && rm /mnt/vast-ai/.probe

echo '=== Step 4: VAST — Is cluster healthy? ==='
vms cluster show
vms alerts list --severity CRITICAL

echo '=== Step 5: VAST — Are all CNodes online? ==='
vms cnodes list

echo '=== Step 6: VAST — Are all DBoxes online? ==='
vms dboxes list

echo '=== Step 7: VAST — Recent errors? ==='
vms events list --severity ERROR --from 2026-08-09T00:00:00
```

✅ **EXPECTED OUTPUT: All steps return OK = storage healthy | First failing step = location of problem**

Support Bundle (For Escalation to VAST Support)

🎯 **PURPOSE: Create a support bundle when the issue needs escalation. This packages all logs for VAST support team.**

```
# Create support bundle with reason
vms supportbundles create --reason 'Performance degradation on training cluster'

# List available support bundles
vms supportbundles list

# Download support bundle (id from list above)
vms supportbundles download --id 1 --output /tmp/vast_bundle.tar.gz

# Send to VAST support
# Upload via VAST support portal or as instructed by VAST support team
```

✅ **EXPECTED OUTPUT: Bundle created | id: N | size: XXX MB | ready for download**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS -X POST https://$VAST_IP/api/supportbundles/ -H 'Content-Type: application/json' -d '{"reason":"performance issue"}' | python3 -m json.tool**

MODULE 7 — Performance Troubleshooting

VAST Admin Role: Diagnose why performance is below expectation. Use VAST metrics to isolate the bottleneck, then fix it.

Performance Metrics for Diagnosis

🎯 **PURPOSE: Pull performance metrics to find where the bottleneck is.**

```
# Get current cluster performance metrics
vms metrics get

# Get metrics for last hour
vms metrics get --from $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%S) \
              --to   $(date -u +%Y-%m-%dT%H:%M:%S)

# Get volume-level metrics to identify hot volumes
vms volumes list

# Show volume details including usage
vms volumes show --name ai_training
```

✅ **EXPECTED OUTPUT: IOPS, throughput MB/s, latency ms — per cluster and per volume**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/monitors/ad_hoc_query/' -X POST -H 'Content-Type: application/json' -d '{"object_type":"cluster","time_frame":"last_hour","prop_list":["rd_iops","wr_iops","rd_bw","wr_bw","rd_lat","wr_lat"]}' | python3 -m json.tool**

Ubuntu: Step-by-Step Throughput Diagnosis

🎯 **PURPOSE: Use Ubuntu tools to confirm where the bottleneck is before checking VAST.**

```
echo '=== Step 1: Check NFS connection count ==='
# If count = 1, nconnect is not set — easiest fix
ss -tn dst :2049 | wc -l | xargs echo 'Connections:'

echo '=== Step 2: Check retransmits ==='
# retrans > 0 = network problem — fix network first
nfsstat -c 2>/dev/null | head -6

echo '=== Step 3: Measure current throughput ==='
fio --name=diag --rw=read --bs=1M --size=512M \
  --directory=/mnt/vast-ai --numjobs=4 --group_reporting \
  2>&1 | grep bw=

echo '=== Step 4: Compare against optimized mount ==='
# If optimized is much faster, remount with those options
fio --name=optimized --rw=read --bs=1M --size=512M \
  --directory=/mnt/vast-ai --numjobs=8 --group_reporting \
  2>&1 | grep bw=
```

✅ **EXPECTED OUTPUT: connections: 16 | retrans=0 | throughput: high MB/s = healthy**

Quota Commands (Capacity Troubleshooting)

🎯 **PURPOSE: Check if a volume has hit its quota limit — jobs fail silently when quota is exhausted.**

```
# List all quotas
vms quotas list

# Show quota details for a specific path
vms quotas show --name ai_training_quota

# Create a quota
vms quotas create \
  --name ai_training_quota \
  --path /ai_training \
  --hard-limit 10TB \
  --soft-limit 9TB

# Modify quota limit
vms quotas modify --name ai_training_quota --hard-limit 20TB

# Delete quota
vms quotas delete --name ai_training_quota
```

✅ **EXPECTED OUTPUT: id | name | path | hard_limit | soft_limit | used | used_percentage**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/quotas/ | python3 -m json.tool**

MODULE 8 — Log Analysis and Diagnostics

VAST Admin Role: Read cluster event logs, build incident timelines, identify root cause from VAST logs.

VAST Event and Alert Log Commands

🎯 **PURPOSE: Extract and analyze cluster events to build an incident timeline.**

```
# List all events (most recent first)
vms events list

# Filter events by severity
vms events list --severity INFO
vms events list --severity WARNING
vms events list --severity ERROR
vms events list --severity CRITICAL

# Filter events by time range
vms events list --from 2026-08-09T00:00:00 --to 2026-08-09T23:59:59

# List resolved alerts
vms alerts list --resolved true

# List active (unresolved) alerts
vms alerts list --resolved false

# Show specific alert details
vms alerts show --id 1
```

✅ **EXPECTED OUTPUT: id | severity | title | description | affected_object | created_at | resolved_at**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS 'https://$VAST_IP/api/events/?severity=ERROR' | python3 -m json.tool**

Ubuntu: Build Incident Timeline

🎯 **PURPOSE: Combine VAST cluster logs with Ubuntu client logs to build a complete incident timeline.**

```
# Step 1: Get VAST events around incident time
# Run from management node or via SSH
vms events list --from 2026-08-09T10:00:00 --to 2026-08-09T12:00:00

# Step 2: On Ubuntu client — check kernel NFS messages
dmesg -T | grep -iE 'nfs|error|fail|timeout' | tail -20

# Step 3: Check Ubuntu system log for NFS errors
sudo grep -i nfs /var/log/syslog | tail -20

# Step 4: Build merged timeline from Ubuntu side
dmesg -T | grep -iE 'nfs|error|fail' | sort > /tmp/ubuntu_timeline.txt
cat /tmp/ubuntu_timeline.txt

# Step 5: Download VAST support bundle for full cluster logs
```

```
vms supportbundles create --reason 'Incident investigation 2026-08-09'
vms supportbundles list
vms supportbundles download --id 1 --output /tmp/vast_incident_bundle.tar.gz
```

✅ **EXPECTED OUTPUT: Merged timeline showing: Ubuntu NFS errors | VAST cluster events | correlated timestamps**

Snapshot Commands (For Recovery After Incidents)

🎯 **PURPOSE: Use snapshots to recover data after accidental deletion or corruption.**

```
# List all snapshots
vms snapshots list

# Create a manual snapshot before risky operation
vms snapshots create \
  --name pre_maintenance_snap \
  --path /ai_training

# Show snapshot details
vms snapshots show --name pre_maintenance_snap

# List snapshot policies
vms snapshotpolicies list

# Create automatic snapshot policy
vms snapshotpolicies create \
  --name daily_snap \
  --every DAY \
  --keep-count 7

# Attach snapshot policy to volume
vms volumes modify --name ai_training --snapshot-policy daily_snap

# Delete a snapshot
vms snapshots delete --name pre_maintenance_snap
```

✅ **EXPECTED OUTPUT: id | name | path | created_at | size | for each snapshot**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/snapshots/ | python3 -m json.tool**

MODULE 9 — Monitoring Fundamentals

VAST Admin Role: Set up continuous monitoring. Configure alert policies. Build a daily health check routine.

Daily Health Check Commands

🎯 **PURPOSE: Run these every morning. All green = cluster healthy. Any red = investigate immediately.**

```
echo '=============================='
echo ' VAST Morning Health Check'
echo '=============================='

echo ''
echo '[1] Cluster Status'
vms cluster show

echo ''
echo '[2] Active Alerts'
vms alerts list --resolved false

echo ''
echo '[3] CNode Health'
vms cnodes list

echo ''
echo '[4] DBox Health'
vms dboxes list

echo ''
echo '[5] Recent Error Events'
vms events list --severity ERROR

echo ''
echo '[6] Performance Metrics'
vms metrics get

echo '=============================='
echo ' Health Check Complete'
echo '=============================='
```

✅ **EXPECTED OUTPUT: Cluster ONLINE | No alerts | All CNodes ONLINE | All DBoxes ONLINE | No errors**

User and Access Management

🎯 **PURPOSE: Manage who can access which volumes. Essential for multi-team environments.**

```
# List all VAST users
vms users list

# Show specific user
vms users show --name admin

# Create a new user
```

```
vms users create --name team_a_user --password SecurePass123

# List user groups
vms usergroups list

# Create user group for AI team
vms usergroups create --name ai_team

# Add user to group
vms usergroups modify --name ai_team --add-users team_a_user
```

✅ **EXPECTED OUTPUT: id | name | email | role | for each user**

🌐 **REST API ALTERNATIVE: curl -sk -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/users/ | python3 -m json.tool**

Capacity Monitoring

🎯 **PURPOSE: Monitor storage capacity continuously. Alert before it fills up.**

```
# Check cluster capacity on VAST
vms cluster show

# Check per-volume usage
vms volumes list

# Check quota usage
vms quotas list

# From Ubuntu: check mounted capacity
df -h /mnt/vast-ai
df -h /mnt/vast-ai | awk 'NR==2{print $5}' | tr -d '%' | xargs -I{} sh -c '[ {} -gt 80 ] && echo ALERT: {}% || echo OK: {}%'

# Find largest directories on VAST from Ubuntu
du -sh /mnt/vast-ai/* 2>/dev/null | sort -rh | head -10
```

✅ **EXPECTED OUTPUT: Cluster: total TB | used TB | available TB | per volume usage | quota used%**

VAST REST API Monitoring Script from Ubuntu

🎯 **PURPOSE: Poll VAST REST API from Ubuntu for continuous monitoring without vms CLI on every node.**

```
# Save as /usr/local/bin/vast_monitor.sh on Ubuntu
# Run every 5 minutes via cron

VAST_IP='<your-cluster-ip>'
VAST_USER='admin'
VAST_PASS='<password>'
LOGFILE='/var/log/vast_health.log'
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

# Check cluster via REST API
HTTP=$(curl -sk -o /dev/null -w '%{http_code}' \
  -u $VAST_USER:$VAST_PASS https://$VAST_IP/api/clusters/)
```

```
if [ "$HTTP" = '200' ]; then
  echo "$TIMESTAMP OK: VAST API reachable" >> $LOGFILE
else
  echo "$TIMESTAMP CRITICAL: VAST API returned HTTP $HTTP" >> $LOGFILE
fi

# Check NFS mount from Ubuntu
touch /mnt/vast-ai/.health 2>/dev/null \
  && echo "$TIMESTAMP OK: NFS write healthy" >> $LOGFILE \
  && rm /mnt/vast-ai/.health \
  || echo "$TIMESTAMP CRITICAL: NFS write failed" >> $LOGFILE

# Check capacity via Ubuntu df
USED=$(df /mnt/vast-ai 2>/dev/null | awk 'NR==2{print $5}' | tr -d '%')
[ -n "$USED" ] && [ "$USED" -gt 80 ] \
  && echo "$TIMESTAMP WARNING: capacity at ${USED}%" >> $LOGFILE \
  || echo "$TIMESTAMP OK: capacity at ${USED}%" >> $LOGFILE
```

✅ **EXPECTED OUTPUT: Log entries: timestamps + OK/WARNING/CRITICAL for each check**

Schedule Monitoring via Cron on Ubuntu

```
# Make script executable
sudo chmod +x /usr/local/bin/vast_monitor.sh

# Schedule every 5 minutes
echo '*/5 * * * * root /usr/local/bin/vast_monitor.sh' | sudo tee
/etc/cron.d/vast_monitor

# View monitoring log
sudo tail -f /var/log/vast_health.log

# View last 20 entries
sudo tail -20 /var/log/vast_health.log
```

✅ **EXPECTED OUTPUT: Log updating every 5 minutes | All OK entries = cluster healthy**

VAST COMMAND QUICK REFERENCE — All 9 Modules

Module	VAST Command	Purpose
1 Architecture	vms cluster show	Full cluster overview
1 Architecture	vms cnodes list	List all compute nodes
1 Architecture	vms dboxes list	List all storage enclosures
1 Architecture	vms volumes list	List all data volumes
1 Architecture	vms volumes create --name X --path /X	Create new volume
1 Architecture	vms views list	List NFS/SMB/S3 exports
1 Architecture	vms views create --name X --path /X --protocols NFS	Create NFS export
2 Performance	vms metrics get	Get cluster performance metrics
2 Performance	vms qospolicies list	List QoS policies
2 Performance	vms qospolicies create --name X --max-throughput-mbps N	Create QoS policy
2 Performance	vms views modify --name X --qos-policy Y	Attach QoS to view
3 Network	vms cnodes show --id N	Show CNode IP and NIC details
3 Network	vms nics list	List all network interfaces
4 Protocol	vms views create --protocols NFSv3	Create NFSv3-only view
4 Protocol	vms views create --protocols S3	Create S3 object view
4 Protocol	vms audits list	View access audit log
4 Protocol	vms s3usergroups list	List S3 access groups
5 Optimization	vms protectionpolicies list	List data reduction policies
5 Optimization	vms protectionpolicies create --compression false	Disable compression (AI files)
5 Optimization	vms volumes modify --name X --protection-policy Y	Attach protection policy
6 Troubleshoot	vms alerts list --severity CRITICAL	Show critical alerts
6 Troubleshoot	vms events list --severity ERROR	Show error events
6 Troubleshoot	vms drives list	List all SSDs
6 Troubleshoot	vms supportbundles create --reason 'text'	Create support bundle
7 Perf Debug	vms metrics get	Get performance data
7 Perf Debug	vms quotas list	Check quota usage
7 Perf Debug	vms quotas modify --name X --hard-limit NTB	Increase quota
8 Logs	vms events list --severity ERROR	Error event log
8 Logs	vms alerts list --resolved false	Active alerts
8 Logs	vms snapshots create --name X --path /Y	Create snapshot before risky work
8 Logs	vms snapshotpolicies create --name X --every DAY	Automated daily snapshots
9 Monitoring	vms alerts list	All active alerts
9 Monitoring	vms users list	List all users
9 Monitoring	vms quotas list	Check all quota usage

REST API Quick Reference — Use When vms CLI Not Available

Resource	REST API Endpoint	Method
Cluster	GET /api/clusters/	curl -sk -u user:pass https://IP/api/clusters/
CNodes	GET /api/cnodes/	curl -sk -u user:pass https://IP/api/cnodes/
DBoxes	GET /api/dboxes/	curl -sk -u user:pass https://IP/api/dboxes/
Volumes	GET /api/folders/	curl -sk -u user:pass https://IP/api/folders/
Views	GET /api/views/	curl -sk -u user:pass https://IP/api/views/
QoS	GET /api/qospolicies/	curl -sk -u user:pass https://IP/api/qospolicies/
Alerts	GET /api/alerts/	curl -sk -u user:pass https://IP/api/alerts/
Events	GET /api/events/	curl -sk -u user:pass https://IP/api/events/
Metrics	POST /api/monitors/ad_hoc_query/	curl -sk -X POST with JSON body
Snapshots	GET /api/snapshots/	curl -sk -u user:pass https://IP/api/snapshots/
Quotas	GET /api/quotas/	curl -sk -u user:pass https://IP/api/quotas/

Users	GET /api/users/	curl -sk -u user:pass https://IP/api/users/
Support Bundle	POST /api/supportbundles/	curl -sk -X POST with reason JSON

💡 **KEY LESSON: When in doubt, use the REST API — it always works on port 443. Every vms command has a REST equivalent. Use 'vms <command> --help' to see all flags for any command.**

⚠️ **IMPORTANT: IMPORTANT: Replace all <placeholders> with real values before running any command. <mgmt-ip> = management node IP. <cnode-ip> = any CNode IP. <vast-cluster-ip> = cluster VIP. Get these from your VAST team.**